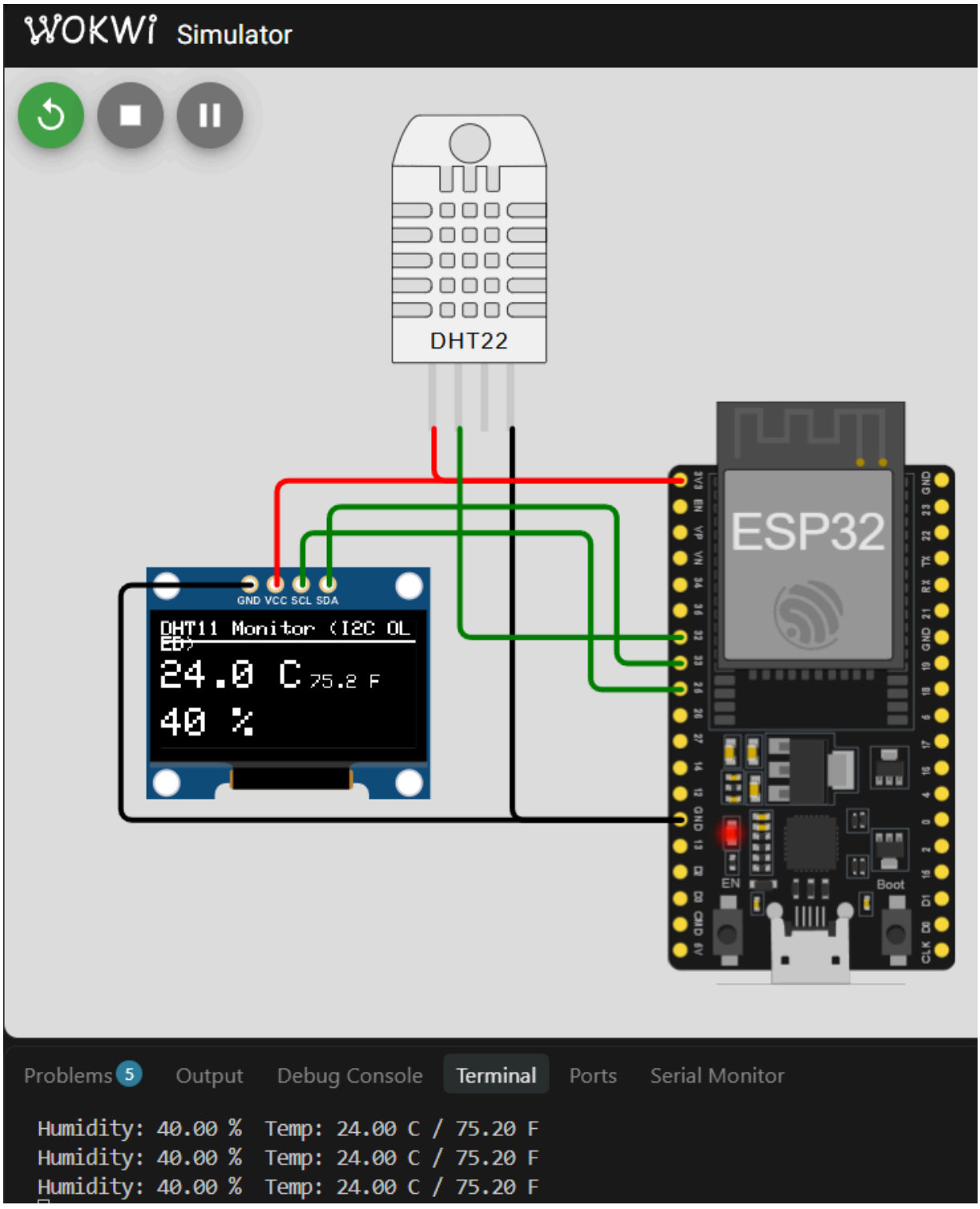
Name: Aaron Ludwig A. Altar **ESP Activity#3**

Section: CPE4B **Date: Sept. 18, 2026**

4.1 DHT11 Monitoring Using OLED Display

Wokwi Simulation Screenshot



Name: Aaron Ludwig A. Altar **ESP Activity#3**

Section: CPE4B **Date: Sept. 18, 2026**

Code Snippet

```
// Activity #4
// 4.1 DHT11 Monitoring Using OLED Display

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// ---------- Pin / Device Definitions ----------
#define DHT_PIN   32
// #define WOKWI_SIM

#ifdef WOKWI_SIM
  #define DHT_TYPE  DHT22
#else
  #define DHT_TYPE  DHT11
#endif

#define SCREEN_WIDTH   128
#define SCREEN_HEIGHT  64
#define OLED_RESET     -1     // No dedicated reset pin (shares ESP32
reset)
#define SCREEN_ADDRESS 0x3C  // Change to 0x3D if display stays blank

// ---------- Timing ----------
const unsigned long READ_INTERVAL_MS = 2000; // DHT11 min. sampling
interval
unsigned long lastReadTime = 0;

// ---------- Objects ----------
DHT dht(DHT_PIN, DHT_TYPE);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

void setup() {
  Serial.begin(115200);
  delay(200);

  dht.begin();
```

Name: Aaron Ludwig A. Altar **ESP Activity#3**

Section: CPE4B **Date: Sept. 18, 2026**

```
  // Initialize I2C on the default ESP32 pins (SDA=21, SCL=22)
  Wire.begin(33, 25);

  if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    Serial.println(F("OLED init failed - check wiring/address."));
    while (true) { delay(1000); } // halt: nothing useful to do without
display
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println(F("DHT11 + OLED Monitor"));
  display.println(F("Initializing..."));
  display.display();
  delay(1000);
}

void loop() {
  unsigned long now = millis();

  if (now - lastReadTime >= READ_INTERVAL_MS) {
    lastReadTime = now;

    float humidity    = dht.readHumidity();
    float temperatureC = dht.readTemperature();        // Celsius
    float temperatureF = dht.readTemperature(true);    // Fahrenheit

    if (isnan(humidity) || isnan(temperatureC)) {
      Serial.println(F("Failed to read from DHT11 sensor."));
      showError();
      return;
    }

    // Log to Serial Monitor
    Serial.print(F("Humidity: "));
    Serial.print(humidity);
```

```
    Serial.print(F(" %  Temp: "));
    Serial.print(temperatureC);
    Serial.print(F(" C / "));
    Serial.print(temperatureF);
    Serial.println(F(" F"));

    showReadings(temperatureC, temperatureF, humidity);
  }
}

void showReadings(float tempC, float tempF, float hum) {
  display.clearDisplay();

  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println(F("DHT11 Monitor (I2C OLED)"));
  display.drawLine(0, 10, SCREEN_WIDTH, 10, SSD1306_WHITE);

  display.setTextSize(2);
  display.setCursor(0, 20);
  display.print(tempC, 1);
  display.print(F(" C"));

  display.setCursor(0, 44);
  display.print(hum, 0);
  display.print(F(" %"));

  display.setTextSize(1);
  display.setCursor(75, 27);
  display.print(tempF, 1);
  display.print(F(" F"));

  display.display();
}

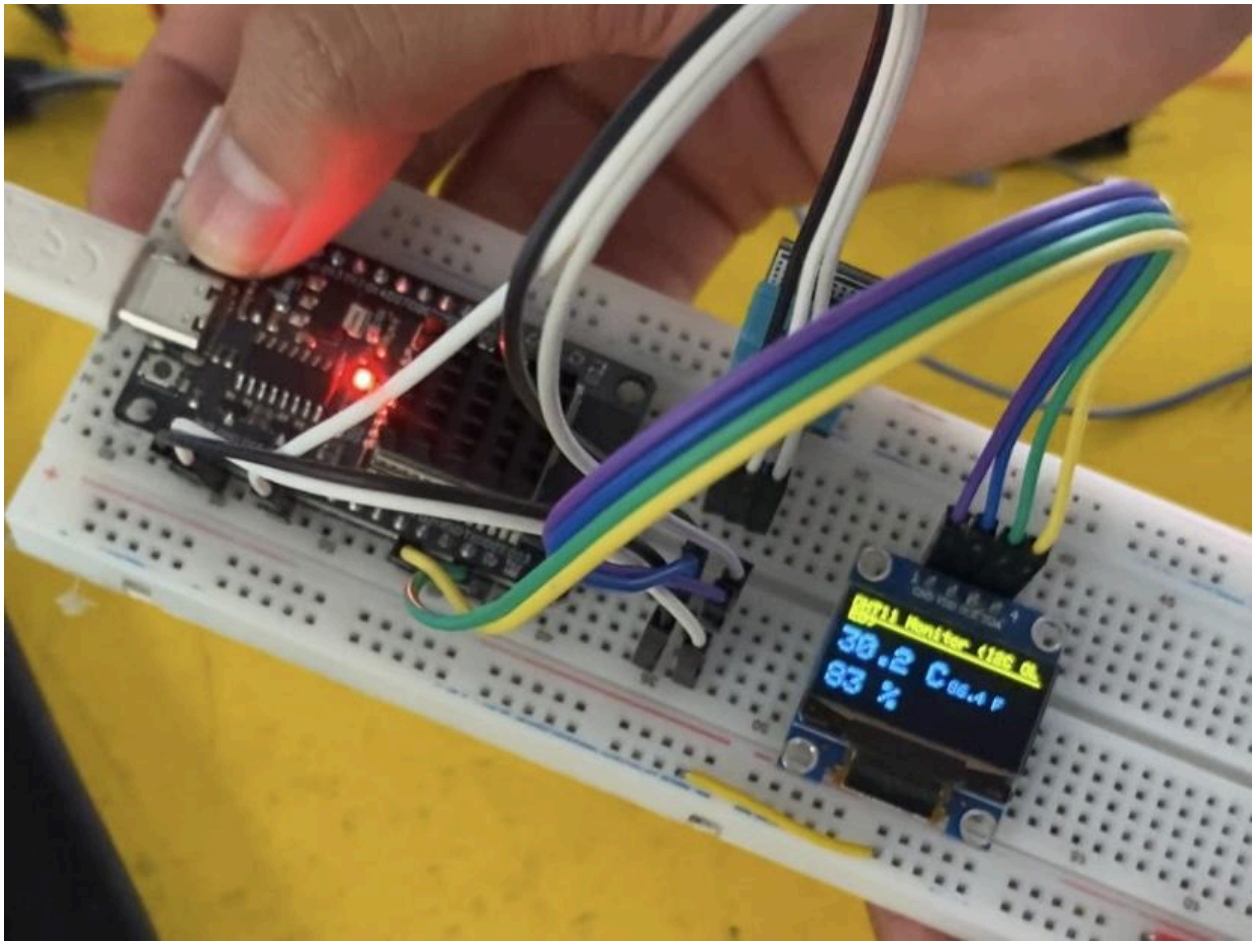
void showError() {
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
```

Name: Aaron Ludwig A. Altar **ESP Activity#3**

Section: CPE4B **Date: Sept. 18, 2026**

```
  display.println(F("DHT11 Monitor"));
  display.drawLine(0, 10, SCREEN_WIDTH, 10, SSD1306_WHITE);
  display.setCursor(0, 25);
  display.println(F("Sensor read error!"));
  display.println(F("Check wiring."));
  display.display();
}
```

Actual Experiment



Video Link:

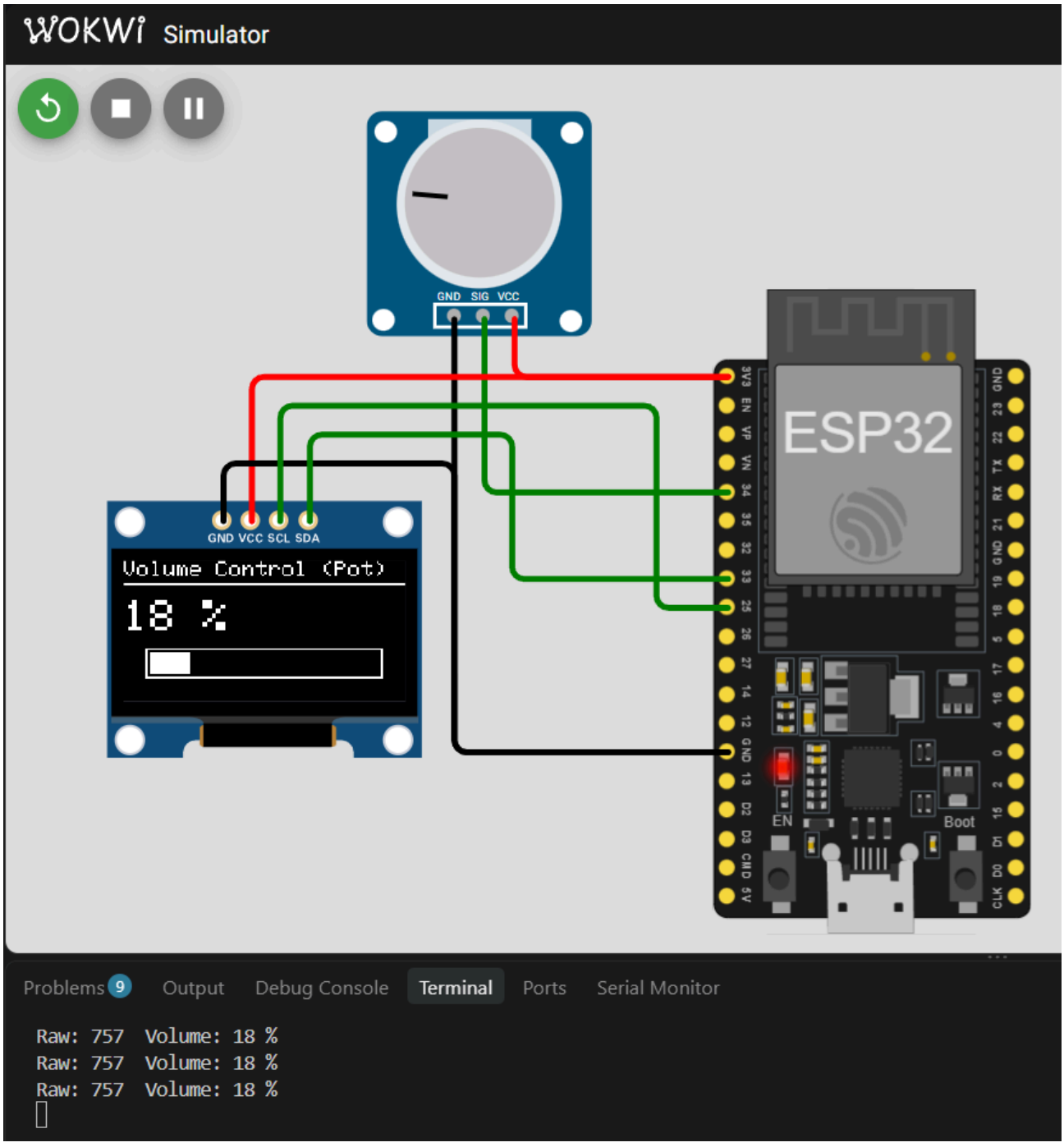
[4.1 DHT11 Monitoring OLED Video Link]

Name: Aaron Ludwig A. Altar **ESP Activity#3**

Section: CPE4B **Date: Sept. 18, 2026**

4.2 Volume Control Using OLED Display

Wokwi Simulation Screenshot



Name: Aaron Ludwig A. Altar **ESP Activity#3**

Section: CPE4B **Date: Sept. 18, 2026**

Code Snippet

```
// Activity #4
// 4.2 Volume Control Using OLED Display

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ---------- Pin Definitions ----------
#define POT_PIN 34   // Potentiometer wiper (ADC1_CH6)

#define SCREEN_WIDTH   128
#define SCREEN_HEIGHT  64
#define OLED_RESET     -1     // No dedicated reset pin (shares ESP32
reset)
#define SCREEN_ADDRESS 0x3C   // Change to 0x3D if display stays blank

// ---------- Bar graph layout ----------
const int BAR_X      = 10;
const int BAR_Y       = 40;
const int BAR_WIDTH  = 108;
const int BAR_HEIGHT = 14;

// ---------- Objects ----------
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

void setup() {
  Serial.begin(115200);
  delay(200);

  pinMode(POT_PIN, INPUT);

  // Initialize I2C on the default ESP32 pins (SDA=21, SCL=22)
  Wire.begin(33, 25);

  if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    Serial.println(F("OLED init failed - check wiring/address."));
    while (true) { delay(1000); } // halt: nothing useful to do without
display
```

```
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println(F("Volume Control Demo"));
  display.println(F("Initializing..."));
  display.display();
  delay(1000);
}

void loop() {
  int rawValue = analogRead(POT_PIN);            // 0-4095
  int volume   = map(rawValue, 0, 4095, 0, 100);  // 0-100 %
  volume = constrain(volume, 0, 100);

  Serial.print(F("Raw: "));
  Serial.print(rawValue);
  Serial.print(F("  Volume: "));
  Serial.print(volume);
  Serial.println(F(" %"));

  showVolume(volume);

  delay(100); // small delay for a stable, readable display update rate
}

void showVolume(int volume) {
  display.clearDisplay();

  // Header
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println(F("Volume Control (Pot)"));
  display.drawLine(0, 10, SCREEN_WIDTH, 10, SSD1306_WHITE);

  // Big percentage readout
  display.setTextSize(2);
```

Name: Aaron Ludwig A. Altar **ESP Activity#3**

Section: CPE4B **Date: Sept. 18, 2026**

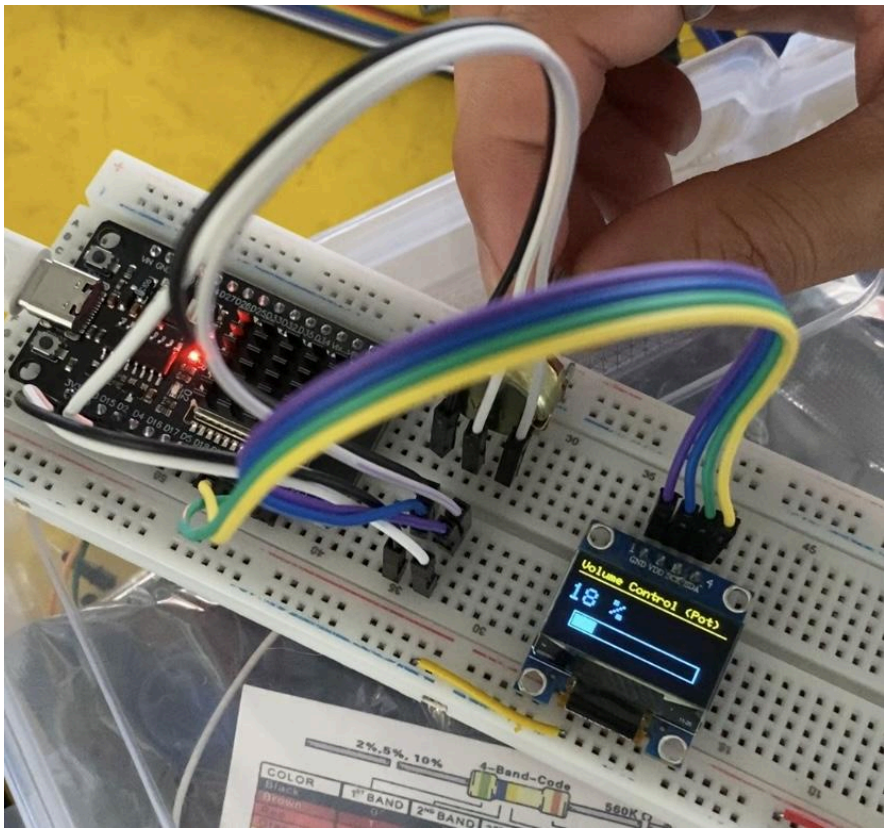
```
  display.setCursor(0, 18);
  display.print(volume);
  display.print(F(" %"));

  // Bar graph outline
  display.drawRect(BAR_X, BAR_Y, BAR_WIDTH, BAR_HEIGHT, SSD1306_WHITE);

  // Bar graph fill, proportional to volume
  int fillWidth = map(volume, 0, 100, 0, BAR_WIDTH - 4);
  display.fillRect(BAR_X + 2, BAR_Y + 2, fillWidth, BAR_HEIGHT - 4,
SSD1306_WHITE);

  display.display();
}
```

Actual Experiment



Video Link

[4.2 Volume Control OLED Video Link]